

课程笔记

Adaptive Architecture for Pentest Agents

自适应渗透测试代理架构设计

绿盟科技 AI 小分队

2026 年 5 月 7 日



视频作者/频道：绿盟科技

发布日期：2025 年 5 月 2 日

视频时长：15 分 55 秒

视频链接：<https://www.bilibili.com/video/BV1PQ9YBKEL6/>

目录

1 引言与背景	2
1.1 演讲概览	2
1.2 竞赛背景	2
1.3 本章小结	2
2 自适应渗透测试架构概览	2
2.1 设计哲学	2
2.2 整体架构	3
2.3 开源工具链整合	4
2.4 本章小结	4
3 Harness 执行引擎设计	4
3.1 执行规划与路径选择	4
3.2 目标验证与动态调整	5
3.3 Challenge 评分机制	6
3.4 本章小结	6
3.5 拓展阅读	7
4 状态管理与 Memory 机制	7
4.1 全局 Memory 架构	7
4.2 Observer Agent 设计	7
4.3 RefLoop 反馈循环	8
4.4 状态压缩策略	8
4.5 本章小结	9
5 性能优化与工程实践	9
5.1 Session 压缩策略	9
5.2 与 Cloud Code 的对比分析	10
5.3 工具链 AI-friendly 重写	10
5.4 Token 消耗优化	11
5.5 本章小结	11
6 总结与延伸	11
6.1 演讲核心回顾	11
6.2 知识串联	12
6.3 实践启示	12
6.4 未来方向	13
6.5 本章小结	13

1 引言与背景

1.1 演讲概览

本次演讲来自**绿盟科技 AI 小分队**，主题为 *Adaptive Architecture for Pentest Agents* (自适应渗透测试代理架构)。该团队在 AI 安全竞赛中经历了从第七名到第一名的跨越，演讲中分享了他们在设计高效、稳定、可扩展的渗透测试自动化系统方面的工程实践与架构思考。

核心主题

如何在资源受限、环境复杂、目标多变的渗透测试场景中，构建一个能够**自适应调整、稳定执行、高效管理**状态的 AI 代理系统。

演讲围绕三个核心板块展开：

1. **整体架构设计**：系统的宏观设计与工程边界约束
2. **Harness 执行引擎**：代理执行的核心机制与动态评估
3. **状态管理与优化**：全局 Memory、Observer Agent 与性能压缩策略

1.2 竞赛背景

渗透测试 (Penetration Testing) 是评估系统安全性的重要手段。传统的人工渗透测试虽然精准，但效率低下、成本高昂。随着大语言模型 (LLM) 的兴起，利用 AI 代理自动执行渗透测试任务成为安全领域的前沿方向。

渗透测试的挑战

- **环境复杂性**：目标系统千差万别，网络拓扑、服务配置、防御机制各不相同
- **执行不确定性**：单次扫描或攻击可能因网络波动、服务响应延迟而失败
- **资源消耗**：LLM 每次调用都消耗 Token，长时间运行的任务成本高昂
- **状态管理**：多步骤攻击需要维护复杂的中间状态，传统脚本难以灵活应对

绿盟科技 AI 小分队在本次竞赛中面对的正是这些挑战。他们从上一届的第七名出发，通过系统性的架构重构和工程优化，最终实现了突破。

1.3 本章小结

本章介绍了演讲的整体背景和核心问题域。渗透测试自动化不仅需要强大的 AI 能力，更需要稳健的工程架构来支撑长时间、多步骤、高不确定性的执行过程。

2 自适应渗透测试架构概览

2.1 设计哲学

绿盟科技 AI 小分队的架构设计遵循三个核心原则：

1. **稳定性优先**：渗透测试代理可能在目标网络中运行数小时甚至数天，任何单点故障都可能导致前功尽弃。系统必须在组件故障时能够优雅降级或快速恢复。
2. **工程边界清晰**：明确定义系统的输入边界、输出边界和状态边界，避免无约束的扩散导致不可控的行为。
3. **可扩展性**：架构应支持不同能力的代理、不同规模的网络环境、不同深度的测试策略的灵活组合。

常见误区

许多初学者设计 AI 代理时容易陷入“功能堆砌”的陷阱——不断添加新工具、新策略，却忽略了系统整体的稳定性和可维护性。渗透测试代理不是功能越多越好，而是在关键路径上足够可靠。

2.2 整体架构

系统的整体架构采用分层设计，核心组件包括：

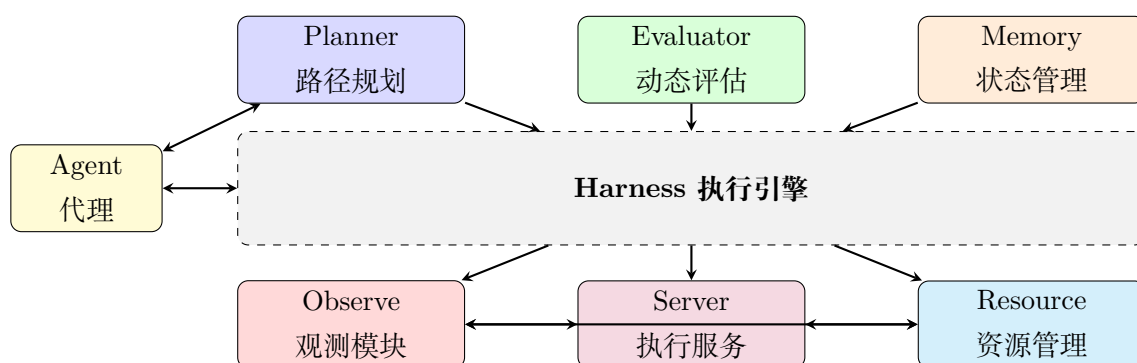


图 1: 自适应渗透测试系统整体架构¹

架构要点

- **Planner**：负责根据当前状态和目标生成执行路径，采用 LOM（Line of Movement）算法进行路径规划
- **Evaluator**：动态评估每一步执行的效果，决定是否继续、回退或切换策略
- **Memory**：维护全局状态，记录执行历史、发现的漏洞、失败的尝试等
- **Harness**：执行引擎的核心，协调各模块协同工作
- **Observe**：持续观测目标环境的状态变化，收集反馈信息
- **Server**：实际执行渗透测试工具的服务层
- **Resource**：管理 Token 预算、时间预算、计算资源等

¹视频画面时间区间：00:01:00-00:01:30。

2.3 开源工具链整合

团队在实践中大量复用了开源社区的成熟工具，而非从零开发。这一策略有两个显著优势：

1. **降低开发成本**：利用现有的 SOAR（Security Orchestration, Automation and Response）平台和渗透测试工具库
2. **提高可靠性**：经过社区验证的工具比新开发的代码更稳定

SOAR 平台

SOAR（安全编排、自动化与响应）是一种将安全工具、流程和人员编排在一起的技术平台。它允许安全团队自动化重复性任务，编排复杂的安全工作流，并快速响应安全事件。在渗透测试场景中，SOAR 平台可以整合扫描器、漏洞利用框架、报告生成工具等。

团队基于开源 SOAR 平台构建了自有的执行层，通过封装和适配，使这些传统工具能够接受 AI 代理的调度指令。

2.4 本章小结

本章介绍了自适应渗透测试系统的整体架构。核心设计思想是：在稳定的工程基础上，通过 Planner、Evaluator、Memory 三个核心模块的协同，实现 AI 代理的智能执行。Harness 作为执行引擎，负责协调各模块的交互，确保系统在有约束的条件下高效运行。

3 Harness 执行引擎设计

3.1 执行规划与路径选择

Harness 引擎的核心职责是将高层级的渗透测试目标分解为可执行的步骤序列。这个过程面临两个关键挑战：

1. **路径爆炸**：渗透测试的攻击面通常很大，从信息收集、漏洞扫描到权限提升、横向移动，可能的执行路径呈指数级增长
2. **动态环境**：目标系统的状态在执行过程中不断变化，静态的规划往往无法适应

LOM 路径规划

团队设计了 **LOM (Line of Movement)** 算法来解决路径选择问题。LOM 不是一次性生成完整的执行计划，而是采用**渐进式规划策略**：

- 每次只规划下一个或下几个步骤
- 每一步执行后根据反馈重新评估和调整后续路径
- 引入**启发式评分**，优先选择信息增益高、成功率高的路径

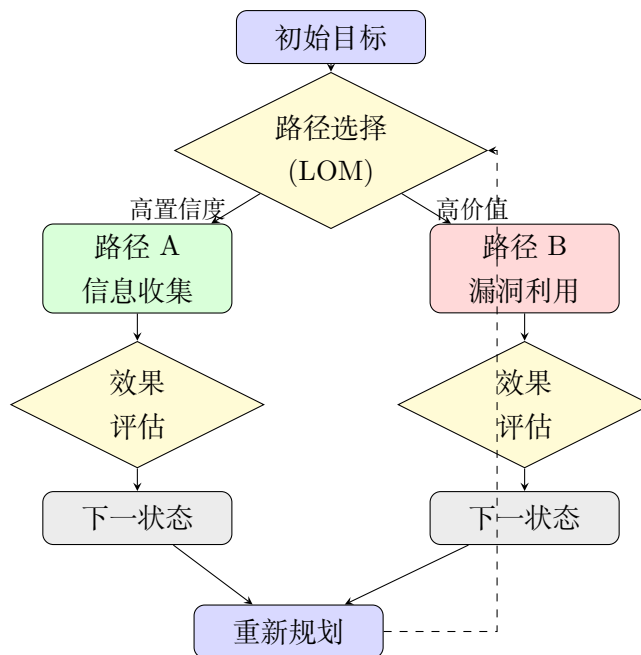


图 2: LOM 渐进式路径规划流程²

3.2 目标验证与动态调整

路径执行后，系统需要验证是否达到了预期目标。团队设计了 **LIM (Line of Interest Monitoring)** 机制：

- **前置验证**：在执行高成本操作前，先验证前提条件是否满足
- **后置验证**：执行后验证目标状态是否达成
- **副作用检测**：检查执行是否产生了预期外的系统变化

验证的重要性

在渗透测试中，**无效的验证**可能导致严重后果：

- 误以为漏洞已修复，实际上利用方式只是变更了
- 重复执行已失败的攻击，浪费 Token 和时间
- 忽略了成功的攻击带来的副作用（如日志记录、服务崩溃）

LIM 机制通过多维度的验证策略，最大程度降低这些风险。

动态调整的核心逻辑可以用以下伪代码表示：

```

1 def execute_with_adaptation(plan, context):
2     for step in plan.steps:
3         # 前置验证
4         if not lim.validate_precondition(step, context):
5             alternative = planner.replan(step, context)
  
```

²视频画面时间区间：00:03:30-00:04:00。

```

6         step = alternative
7
8     # 执行步骤
9     result = harness.execute(step)
10
11    # 后置验证
12    if lim.validate_outcome(step, result, context):
13        memory.commit(step, result)
14        context.update(result)
15    else:
16        # 执行失败，评估是否重试或切换路径
17        if evaluator.should_retry(step, result):
18            plan.insert_retry(step)
19        else:
20            alternative = planner.find_alternative(step, context)
21            plan.replace(step, alternative)
22
23    return context

```

Listing 1: 动态评估与调整逻辑

3.3 Challenge 评分机制

为了在竞赛环境中快速评估代理的表现，团队设计了 **Challenge 模块**。该模块对每一次探测尝试进行评分，帮助系统识别哪些路径是有效的、哪些是无效的。

表 1: Challenge 评分维度

维度	说明	权重
信息增益	是否获取了新的目标信息	高
成功概率	基于历史数据的成功可能性	中
资源消耗	Token 和时间成本	中
副作用风险	对目标系统的影响程度	高
可恢复性	失败后的回退成本	低

评分的工程意义

Challenge 评分不仅用于竞赛排名，更重要的是为 Planner 提供**反馈信号**。评分高的路径会被优先选择，评分低的路径会被降级或排除。这种机制使系统能够快速学习哪些策略在特定类型的目标上更有效。

3.4 本章小结

Harness 执行引擎是自适应渗透测试系统的核心。通过 LOM 渐进式路径规划、LIM 多维度目标验证、Challenge 评分反馈三个机制的配合，系统能够在复杂多变的环境中做出合理的执行决策。

3.5 拓展阅读

- SOAR 平台参考架构: <https://www.gartner.com/en/information-technology/glossary/security-orchestration-automation-response-soar>

4 状态管理与 Memory 机制

4.1 全局 Memory 架构

渗透测试是一个高度状态依赖的过程。每一步的执行都依赖于之前步骤的结果，而错误的中间状态可能导致后续所有步骤的失败。因此，**可靠的状态管理**是整个系统的基石。

团队设计了**全局 Memory 架构**，将系统状态分为以下几个层次：

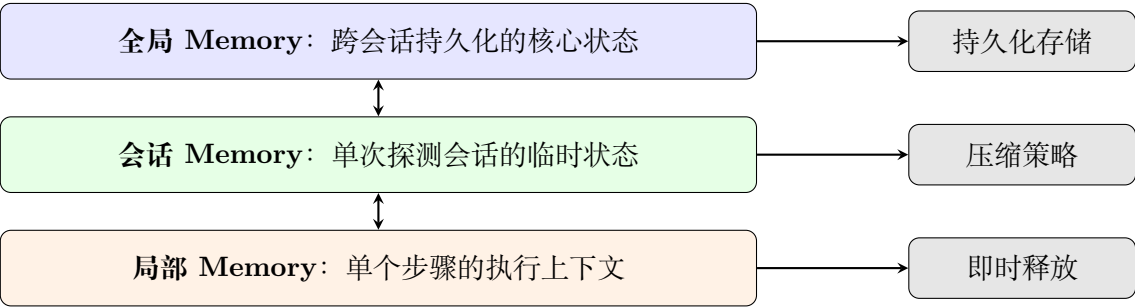


图 3: Memory 三层架构³

Memory 层次说明

- **全局 Memory**：记录已发现的漏洞、目标系统的整体画像、成功的攻击路径等。这些数据在多次探测会话之间共享，是系统“经验积累”的载体。
- **会话 Memory**：单次探测过程中的中间状态，如当前权限级别、已访问的目录、扫描结果等。会话结束后，有价值的内容会被提取到全局 Memory，其余内容被压缩或丢弃。
- **局部 Memory**：单个执行步骤的临时上下文，如函数参数、临时文件路径等。步骤结束后立即释放。

4.2 Observer Agent 设计

Observer Agent 是状态管理的关键组件。它采用 **Sidecar 模式**运行，即与主执行流程并行但独立运行，持续观测系统状态的变化。

Sidecar 模式

Sidecar 模式是一种微服务架构模式，主服务容器（主 Agent）与一个或多个辅助容器（Sidecar）协同工作。Sidecar 容器与主容器共享生命周期，但独立运行，负责日志收集、监控、配置管理等横切关注点。在渗透测试场景中，Observer Sidecar 专门负责状态观测和维护。

Observer Agent 的核心职责包括：

³视频画面时间区间：00:07:00-00:07:30。

- 1. **状态快照**：在关键节点捕获系统状态的完整快照
- 2. **变化检测**：对比前后状态，识别有意义的变化
- 3. **异常告警**：检测到异常状态时及时通知主流程
- 4. **状态修复**：在某些情况下自动修复不一致的状态

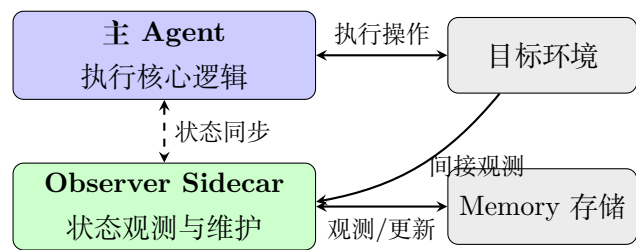


图 4: Observer Sidecar 架构⁴

4.3 RefLoop 反馈循环

RefLoop（Reflection Loop）是系统自我改进的核心机制。它在每次执行周期结束后，对整个 过程进行反思和总结，提炼出可复用的经验。

RefLoop 的工作流程：

- 1. **数据收集**：收集本次执行的所有输入、输出、中间状态和评分
- 2. **模式识别**：识别成功和失败的模式，找出关键影响因素
- 3. **策略更新**：基于分析结果更新 Planner 和 Evaluator 的策略参数
- 4. **知识沉淀**：将新发现的知识写入全局 Memory

RefLoop 的价值

RefLoop 使系统具备**持续学习能力**。即使面对全新的目标环境，系统也能快速适应，因为之 前的经验为它提供了合理的初始假设和探索方向。这种能力在竞赛场景中尤为重要——同一 类型的漏洞往往在不同目标上以相似的形式出现。

4.4 状态压缩策略

长时间运行的渗透测试代理会产生大量的中间状态数据。如果不加以管理，这些数据会迅速耗 尽 Memory 容量。团队设计了多层状态压缩策略：

⁴视频画面时间区间：00:08:30-00:09:00。

表 2: 状态压缩策略

策略	触发条件	效果
摘要压缩	会话结束	将详细日志压缩为结构化摘要
去重合并	定期执行	合并重复的扫描结果
分层丢弃	Memory 接近上限	按重要性丢弃低价值状态
引用替换	全局状态更新	用引用代替冗余复制

压缩的代价

过度压缩可能导致信息丢失。例如，丢弃”失败的尝试”记录意味着系统无法学习到”什么策略不可行”。团队的做法是：对成功路径进行激进压缩（只保留关键节点），对失败路径进行保守压缩（保留更多上下文以便分析失败原因）。

4.5 本章小结

状态管理是自适应渗透测试系统的基石。通过全局 Memory 的三层架构、Observer Agent 的 Sidecar 模式、RefLoop 的自我反思机制，以及智能的状态压缩策略，系统能够在有限的资源下维护足够丰富的状态信息，支持高效、可靠的长期执行。

5 性能优化与工程实践

5.1 Session 压缩策略

在竞赛环境中，Token 消耗直接决定了系统的运行成本。团队通过精细的 Session 压缩策略，在不显著降低效果的前提下大幅减少了 Token 使用量。

核心思路是：区分必要信息和冗余信息。

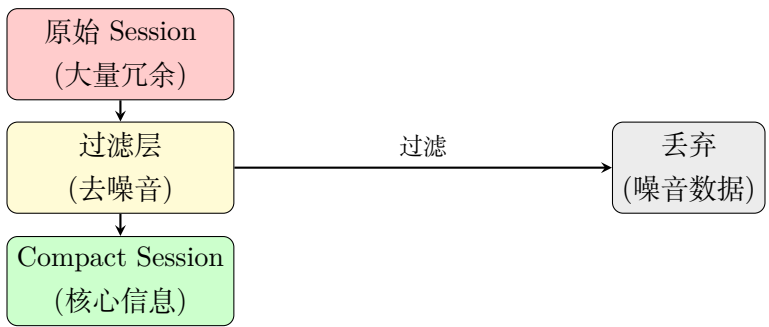


图 5: Session 压缩流程⁵

具体措施包括：

- 1. **去重**：多次扫描同一端口或服务的重复结果只保留最新版本
- 2. **截断**：过长的命令输出只保留前 N 行和最后 N 行（头部和尾部往往最关键）
- 3. **抽象**：将具体的文件路径、IP 地址等替换为符号引用

⁵ 视频画面时间区间：00:11:00–00:11:30。

4. **分层存储**：详细数据存入文件系统，Memory 中只保留索引和摘要

5.2 与 Cloud Code 的对比分析

团队将自有方案与业界的 Cloud Code 方案进行了对比。Cloud Code 是一种将代码执行托管到云端服务的模式，具有资源丰富、环境统一的优势，但也存在延迟高、成本高、灵活性差的问题。

表 3: 自有方案 vs Cloud Code 对比

维度	自有方案	Cloud Code
执行延迟	低（本地/近端执行）	较高（云端往返）
Token 消耗	可控（自定义压缩）	较高（完整上下文传递）
环境灵活性	高（可自定义环境）	中（受限于云服务）
扩展性	高（自主控制资源）	中（依赖云服务商）
运维复杂度	较高（需自行维护）	低（托管服务）

关键洞察

团队的核心发现是：**对于渗透测试这种需要高频交互、快速反馈的场景，本地执行 + 智能压缩的方案在总成本（时间 + Token）上优于云端托管方案。**这并不意味着 Cloud Code 没有价值，而是说明不同的应用场景需要不同的工程权衡。

5.3 工具链 AI-friendly 重写

传统渗透测试工具的输出格式往往是为人类设计的——包含大量格式化文本、颜色代码、进度条等。这些内容对 AI 代理来说是噪音，会浪费宝贵的 Token 预算。

团队对常用工具进行了 **AI-friendly 重写**（称为 RTRewrite），核心改造包括：

- **结构化输出**：将纯文本输出改为 JSON/结构化格式
- **摘要模式**：增加仅输出关键信息的模式
- **增量更新**：只输出变化的部分，而非完整结果
- **语义标注**：为输出内容添加语义标签（如”高危漏洞”、”信息泄露”等）

```
{
  "scan_type": "port_scan",
  "target": "192.168.1.1",
  "summary": {
    "total_ports": 65535,
    "open_ports": 3,
    "key_findings": [
      {"port": 22, "service": "ssh", "risk": "medium"},
      {"port": 80, "service": "http", "risk": "high"},
      {"port": 443, "service": "https", "risk": "medium"}
    ]
  }
},
```

```

13     "details_truncated": true,
14     "full_log_path": "/logs/scan_001.json"
15 }

```

Listing 2: AI-friendly 输出示例（结构化 JSON）

5.4 Token 消耗优化

团队在实践中总结了一套 Token 优化的方法论：

1. **上下文裁剪**：只保留与当前决策最相关的历史信息
2. **分层摘要**：对远距离的历史使用更粗粒度的摘要
3. **缓存复用**：对重复的查询使用缓存结果
4. **模型选择**：简单任务使用小模型，复杂任务使用大模型

表 4: 优化前后 Token 消耗对比

场景	优化前	优化后
单次端口扫描	8K tokens	2K tokens
漏洞分析	15K tokens	4K tokens
路径规划	5K tokens	1.5K tokens
状态更新	3K tokens	0.8K tokens

Token 优化的深层思考

Token 优化不仅仅是“省钱”的问题。更少的 Token 意味着：

- 更快的响应速度（传输和处理时间减少）
- 更长的有效上下文窗口（同样预算下可以保留更多信息）
- 更高的推理质量（去除噪音后，模型能更专注于关键信息）

5.5 本章小结

本章介绍了团队在性能优化方面的工程实践。Session 压缩、AI-friendly 工具重写、Token 优化三位一体的策略，使系统在保证功能完整性的前提下，实现了高效的资源利用。

6 总结与延伸

6.1 演讲核心回顾

绿盟科技 AI 小分队的演讲围绕一个核心问题展开：**如何在复杂多变的渗透测试场景中，构建一个既智能又可靠的 AI 代理系统？**

通过对演讲内容的系统梳理，我们可以将他们的设计思想归纳为以下几个层次：

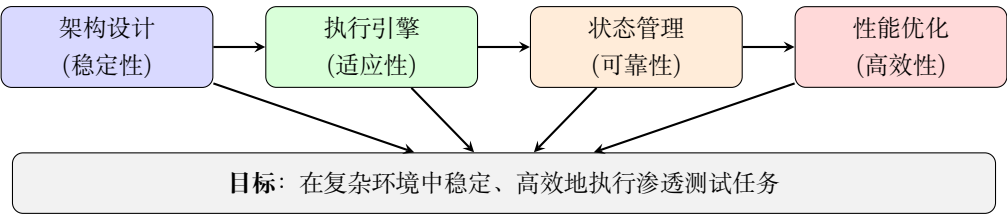
- 1. **架构层**：以稳定性为首要目标，通过清晰的分层架构（Planner-Evaluator-Memory-Harness）实现各模块的职责分离和协同工作。
- 2. **执行层**：采用渐进式规划（LOM）和多维度验证（LIM），使系统能够在不确定环境中做出合理的执行决策。
- 3. **状态层**：通过全局 Memory、Observer Sidecar 和 RefLoop，构建了一个自我维护、自我反思的状态管理系统。
- 4. **优化层**：Session 压缩、AI-friendly 工具重写和 Token 优化，使系统在资源受限的竞赛环境中仍能保持高效运行。

设计思想的本质

绿盟科技 AI 小分队的方案本质上是一种**工程驱动的 AI 架构**——他们不是简单地用更大的模型、更多的算力来解决问题，而是通过精巧的系统设计和工程优化，让有限的 AI 能力在复杂的实际场景中发挥最大效用。这种”架构优于规模”的思路，对于资源受限的 AI 应用场景具有重要的参考价值。

6.2 知识串联

将本章内容与前述章节串联，可以形成一条清晰的设计脉络：



四个层次相辅相成：没有稳定的架构，执行引擎就失去了依托；没有可靠的执行，状态管理就成为空中楼阁；没有高效的状态管理，性能优化就无从谈起。四个层次共同支撑起最终的目标。

6.3 实践启示

基于本次演讲的内容，我们可以提炼出以下对实际工作有指导意义的启示：

- 1. **先工程后智能**：在构建 AI 系统时，先确保工程基础稳固，再叠加智能能力。一个不稳定的系统配上再强的 AI 也无法可靠工作。
- 2. **渐进式优于一次性**：在复杂场景中，渐进式规划（LOM）比一次性规划更鲁棒。因为环境随时可能变化，保留调整空间是明智之举。
- 3. **状态是核心资产**：对于多步骤、长周期的 AI 任务，状态管理的重要性不亚于模型本身。投入精力设计良好的状态架构是值得的。
- 4. **优化要系统思维**：Token 优化不是简单的”少传点内容”，而是要从信息表示、工具设计、上下文管理等多个维度综合考虑。

6.4 未来方向

演讲中也提到了一些未来的优化方向：

- **多 Agent 协作**：当前架构以单 Agent 为主，未来可以探索多 Agent 并行探测、分工协作的模式
- **更细粒度的状态管理**：进一步细化状态的抽象层次，支持更灵活的状态操作
- **强化学习集成**：将 RefLoop 与强化学习结合，实现更自动化的策略优化
- **跨域迁移**：将在渗透测试场景中积累的经验迁移到其他安全测试领域

6.5 本章小结

本章对演讲内容进行了系统性的回顾和延伸思考。绿盟科技 AI 小分队从竞赛实践中提炼出的架构设计思想——稳定性优先、渐进式规划、可靠状态管理、系统化优化——不仅适用于渗透测试场景，对于构建任何需要在复杂环境中长期运行的 AI 系统都具有重要的参考价值。

——全文完——